FINAL PROJECT • GROUP 2

Classification of Hand Gestures

Using Deep Convolutional Neural Networks

MOTIVATION

What is Hand Gesture Recognition?

Teaching computers to see and understand the gestures we make with our hands

Sign Language

Translating gestures into text or speech to bridge communication for deaf and hard-of-hearing communities.

Human-Computer Interaction

Replacing keyboards and mice with natural hand movements, making technology more accessible.

VR & Gaming

Making immersive experiences feel intuitive instead of clunky.

Healthcare

Hands-free control of medical equipment and assistive devices for people with mobility challenges.

The common thread: making technology adapt to humans, not the other way around.

THE CHALLENGES

Why is this so hard?

It looks easy to us, but computers face real challenges.

1 Hand variability

Different sizes, shapes, finger lengths, skin tones, even calluses and scars.

2 Lighting

A fist in shadow looks completely different from a fist in bright sunlight.

3 Backgrounds

Cluttered scenes confuse models trained on clean data.

4 Hand angles

Rotation, perspective, distance from the camera, how much of the wrist is visible.

5 Real-time speed

A model that takes 5 seconds per frame is useless for interaction.

6 Subject generalization

Will the model work for someone it has never seen?

WHAT WE SET OUT TO ANSWER

Our Research Questions

Our core question came from the proposal

CORE QUESTION

Can deep convolutional neural networks classify hand gestures accurately and efficiently?

ARCHITECTURE

Are CNNs really the right tool, or could simpler models work just as well?

COMPONENTS

Which parts of the network architecture actually matter, and which are decorative?

EVALUATION

How much does our choice of evaluation method change the numbers we report?

METHODOLOGY

How We Approached the Project

Entire pipeline built from scratch in PyTorch fully reproducible.

1. Five machine-learning models, from simple linear regression to a deep CNN
2. LeapGestRecog dataset : 20,000 hand images, 10 gestures, 10 different people
3. Evaluation methods to test how much the method itself matters
4. Ablation studies deliberately breaking parts of the best model to see what really matters
5. Grad-CAM visualizations to peek inside the model and see what it focuses on
6. Hyperparameter tuning via grid and random search
7. Every run automatically saves config, training curves, and final results for reproducibility

DATA

The Dataset: LeapGestRecog

20,000	Total images
10	Gesture classes
10	Different people
Near-IR	Imagery type

Why this dataset?

- Captured with a near-infrared sensor , hands appear bright against a dark background, removing most lighting and background variability.
- 10 people performing every gesture gives natural variation across hand shapes and styles.
- Clean enough to train deep models reliably, small enough to fit on a laptop.
- Perfect for a focused study of architectures and evaluation methods, without getting lost in data engineering.

DATA

The 10 Gesture Classes

The model has to distinguish these ten gestures.

palm open hand, fingers spread	**L** thumb up, index pointing sideways	**fist** closed hand	**fist moved** closed hand at a different angle	**thumb** thumbs up
index pointing with the index finger	**ok** thumb and index touching	**palm moved** open hand at a different angle	**c** hand curved like the letter C	**down** pointing downward

Highlighted pairs are visually similar these will be the hardest for any model to tell apart.

METHOD

The Five Models We Compared

A deliberate progression from simplest to most expressive each step adds exactly one capability.

Linear Regression	164,000 params	Single flat layer + MSE loss (wrong loss on purpose)	**Baseline**
Logistic Regression	164,000 params	Same architecture but with cross-entropy loss	**+ Correct loss**
Multi-Layer Perceptron	~4,200,000 params	Adds two hidden layers with ReLU nonlinearity	**+ Nonlinearity**
SmallCNN	5,226 params	Two convolutional blocks first use of spatial structure	**+ Convolutions**
Custom CNN	422,986 params	Four deep conv blocks plus BatchNorm and dropout	**+ Depth**

Comparing them tells us which capability is actually doing the work.

METHOD

How We Trained Everything

All five models trained with identical settings so the comparison is fair.

FRAMEWORK

PyTorch

EPOCHS

10–20, depending on convergence

OPTIMIZER

Adam with small weight decay

LOSS

Cross-entropy (MSE for linear baseline)

LEARNING RATE

Cosine annealing schedule

AUGMENTATION

Flips, small rotations, slight translations

BATCH SIZE

64 images per step

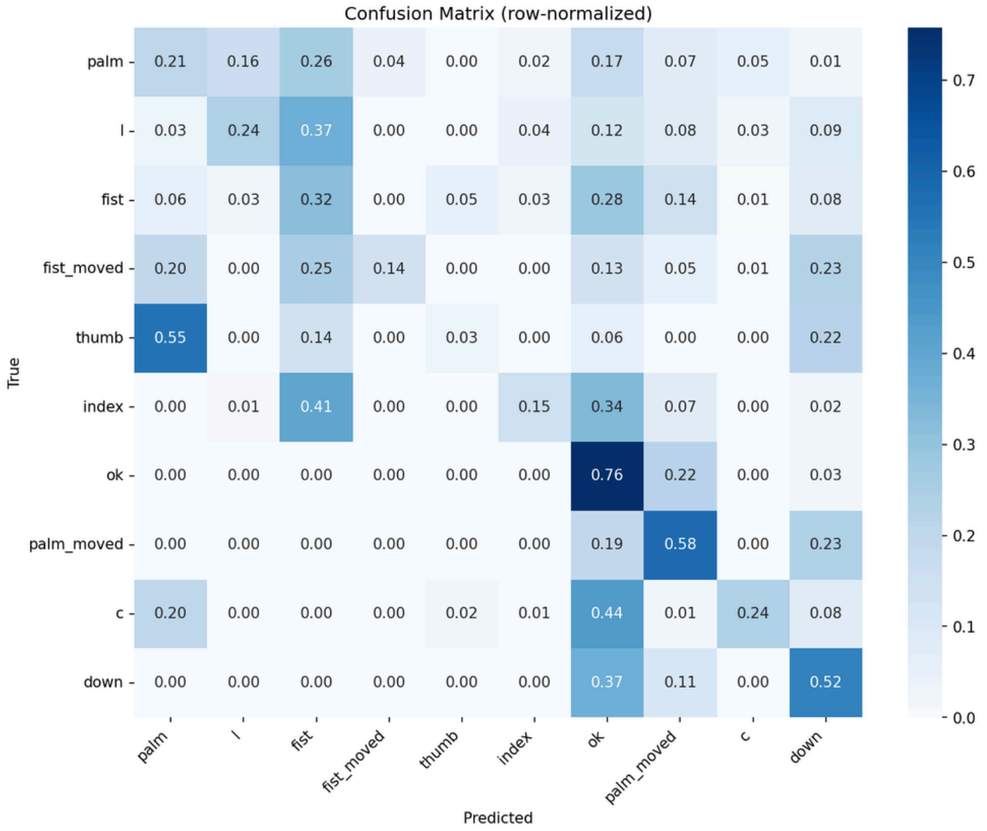
TUNING

Grid + random search on lr, dropout, weight decay

Every run auto-saves its config, training curves, and final test results for full reproducibility.

METHOD

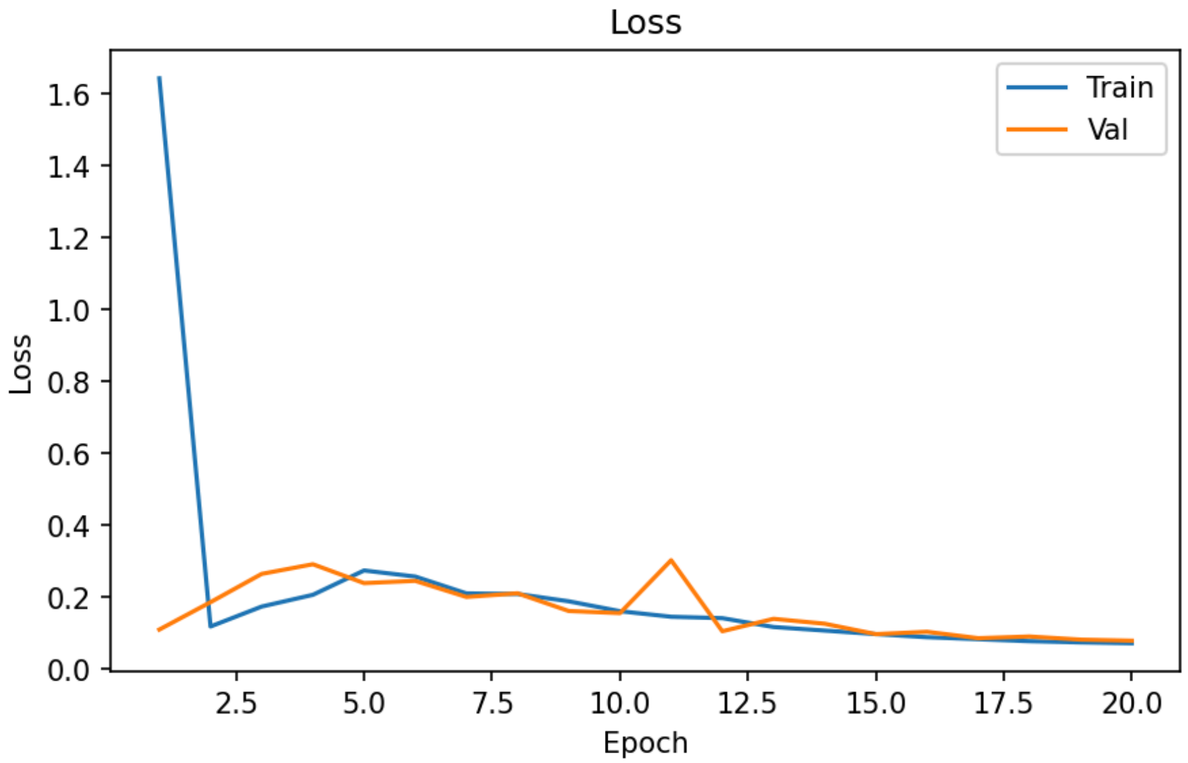
Linear Regression

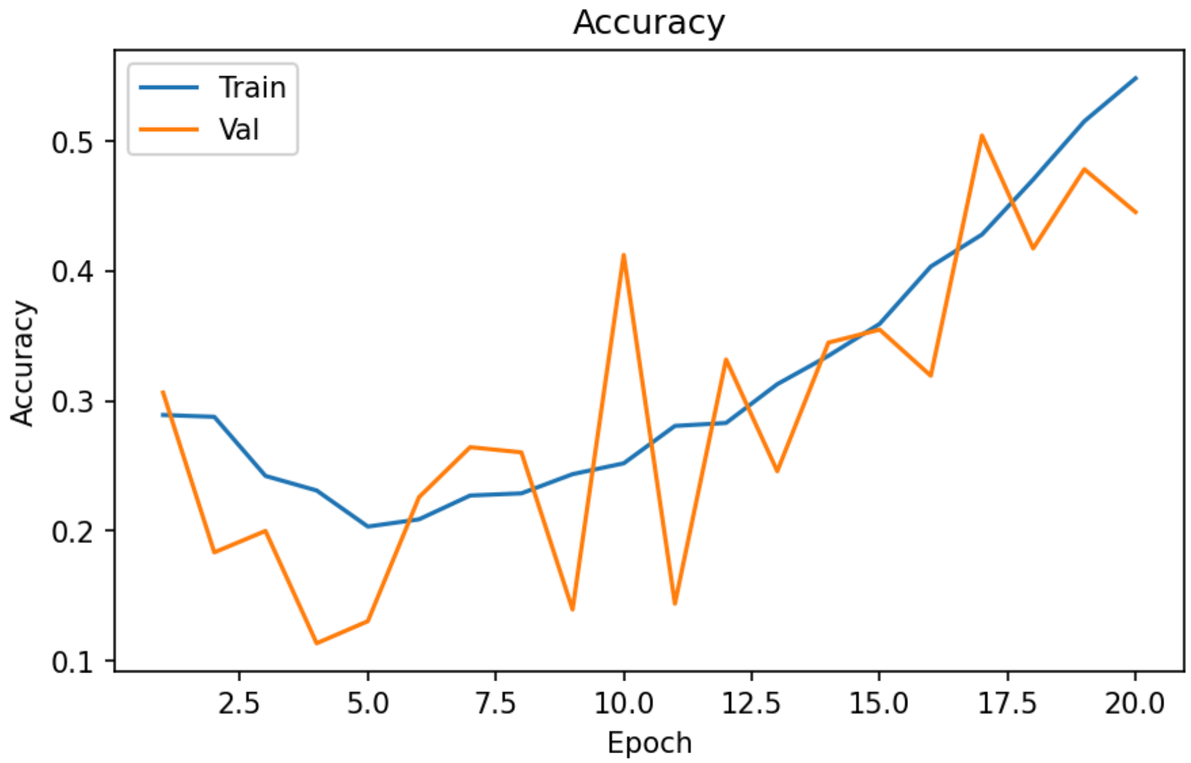


The model averages about 32% accuracy across classes. Diagonal values are weak almost everywhere (palm 0.21, fist 0.32, thumb 0.03). The model uses "ok" as a dumping ground c gets predicted as ok (0.44), index gets predicted as ok (0.34). The strongest classes are ok itself (0.76) and palm_moved (0.58), suggesting the linear decision boundary can only separate a few visually distinct gestures.

METHOD

Linear Regression

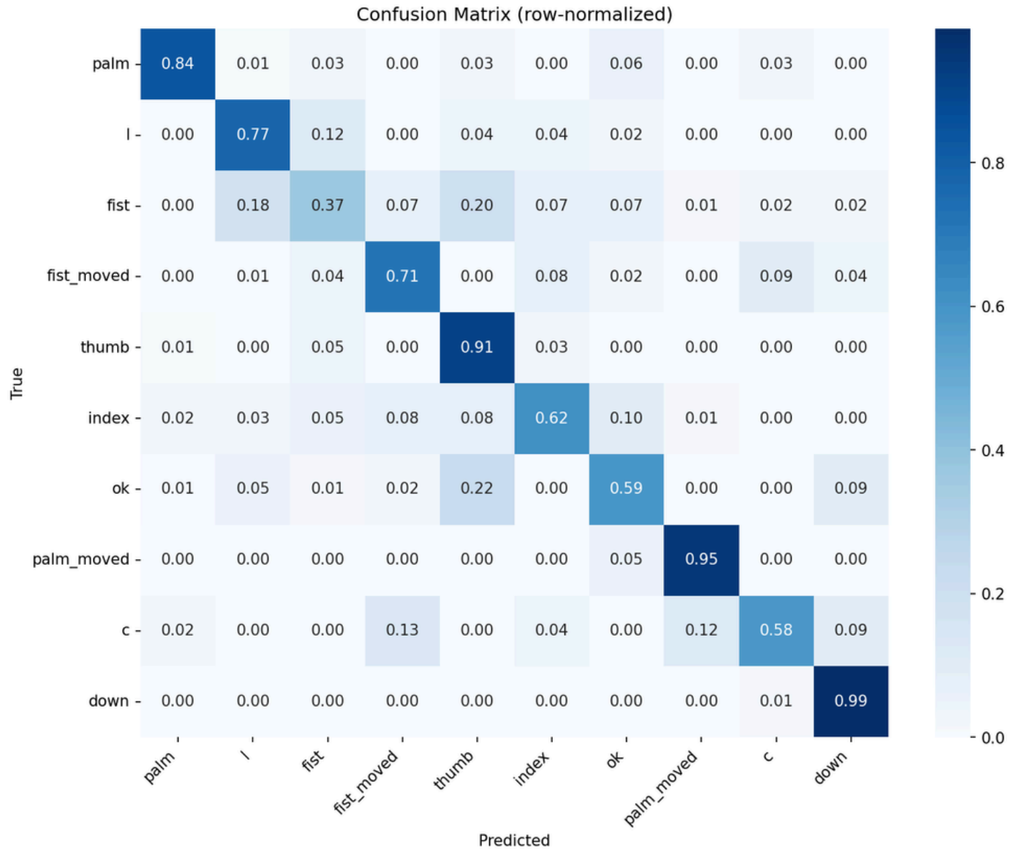




Loss drops fast then plateaus around 0.1 (MSE on one-hot targets, not cross-entropy). Validation accuracy is extremely volatile, swinging between 0.12 and 0.50. This is exactly what we'd expect from regression with the wrong loss function gradients don't push hard enough toward correct class probabilities, so optimization is unstable.

METHOD

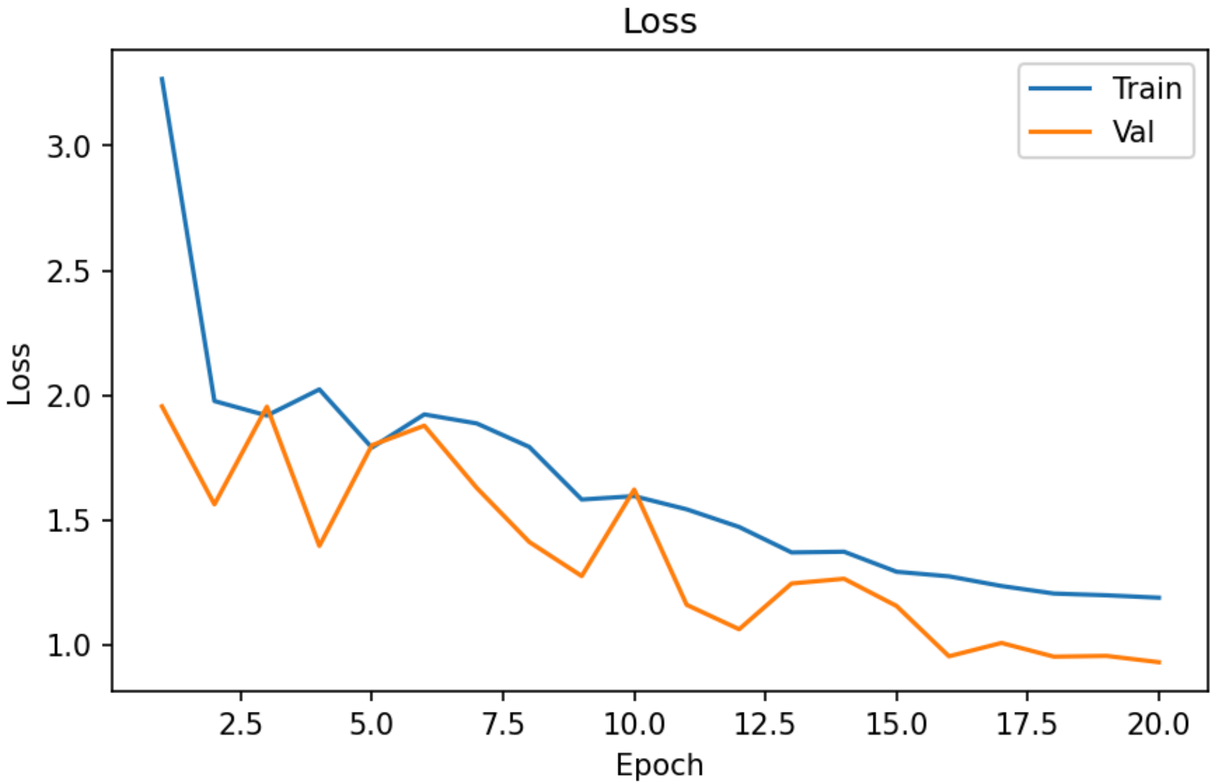
Logistic Regression

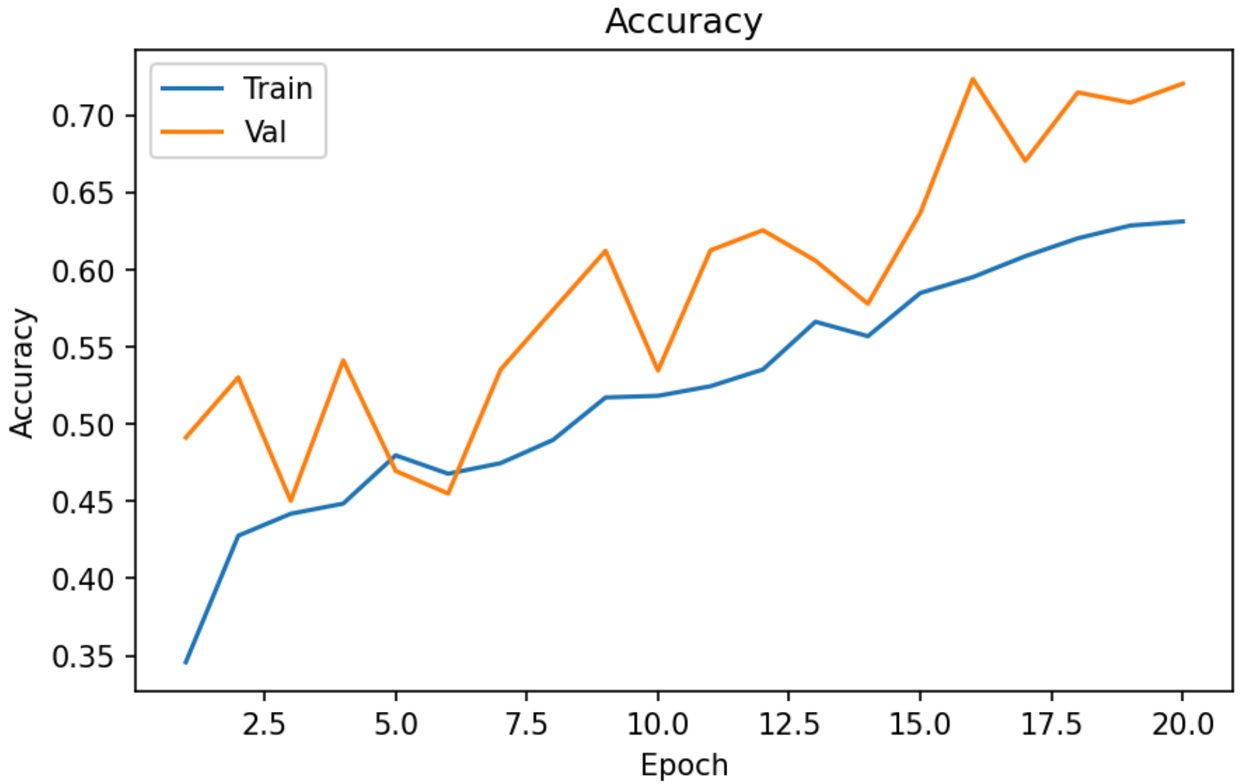


Big jump to ~73% average diagonal. Same architecture as linear, but cross-entropy loss transforms the result. Strong classes: down (0.99), palm_moved (0.95), thumb (0.91), palm (0.84). Weak spot: fist at 0.37 confused with L (0.18), thumb (0.20), fist_moved (0.07). The visually-similar pairs we predicted are exactly the ones struggling.

METHOD

Logistic Regression

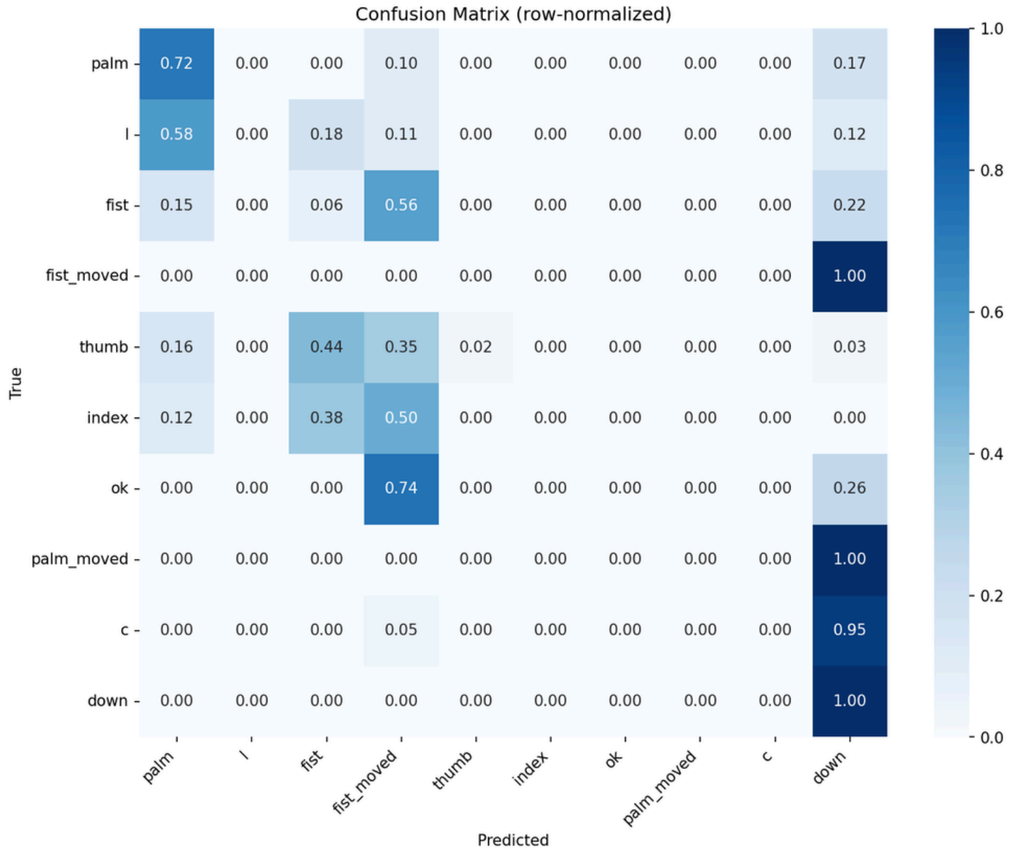




Smooth, healthy convergence. Loss drops from 3.3 → 1.2 (train) and 2.0 → 0.95 (val). Val accuracy reaches 72%, matching the confusion matrix. Interestingly, val outperforms train suggests the validation subject happens to be easier than the average training subject.

METHOD

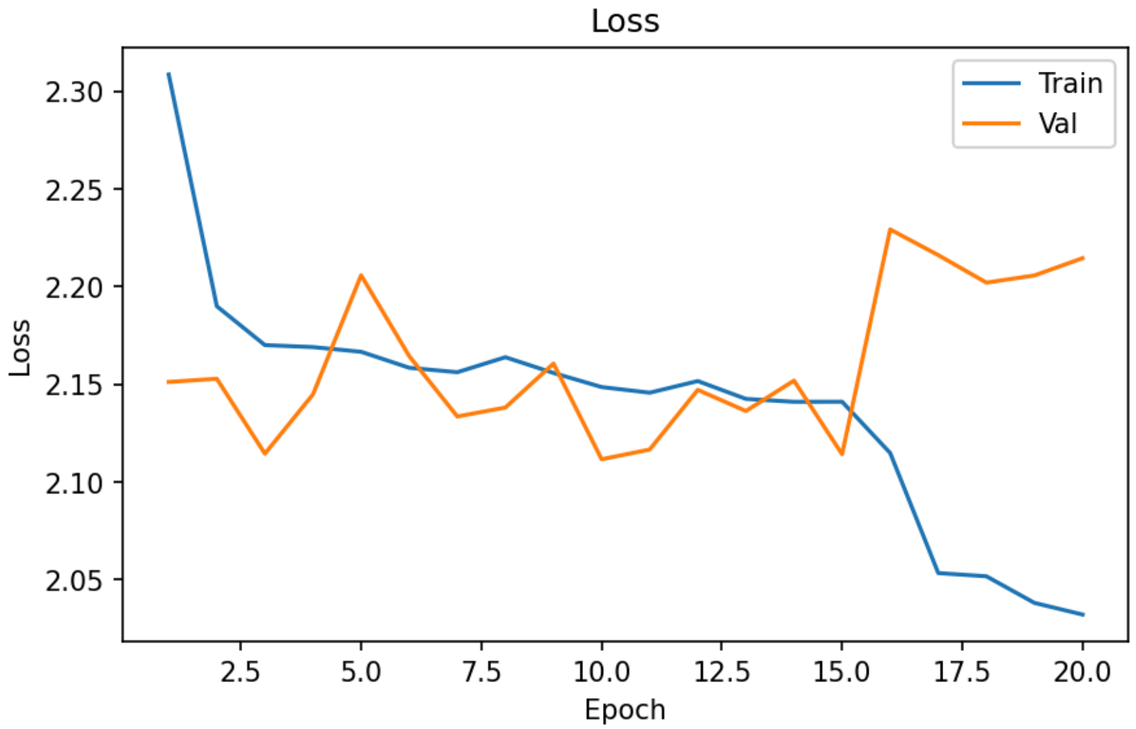
Multi-Layer Perceptron

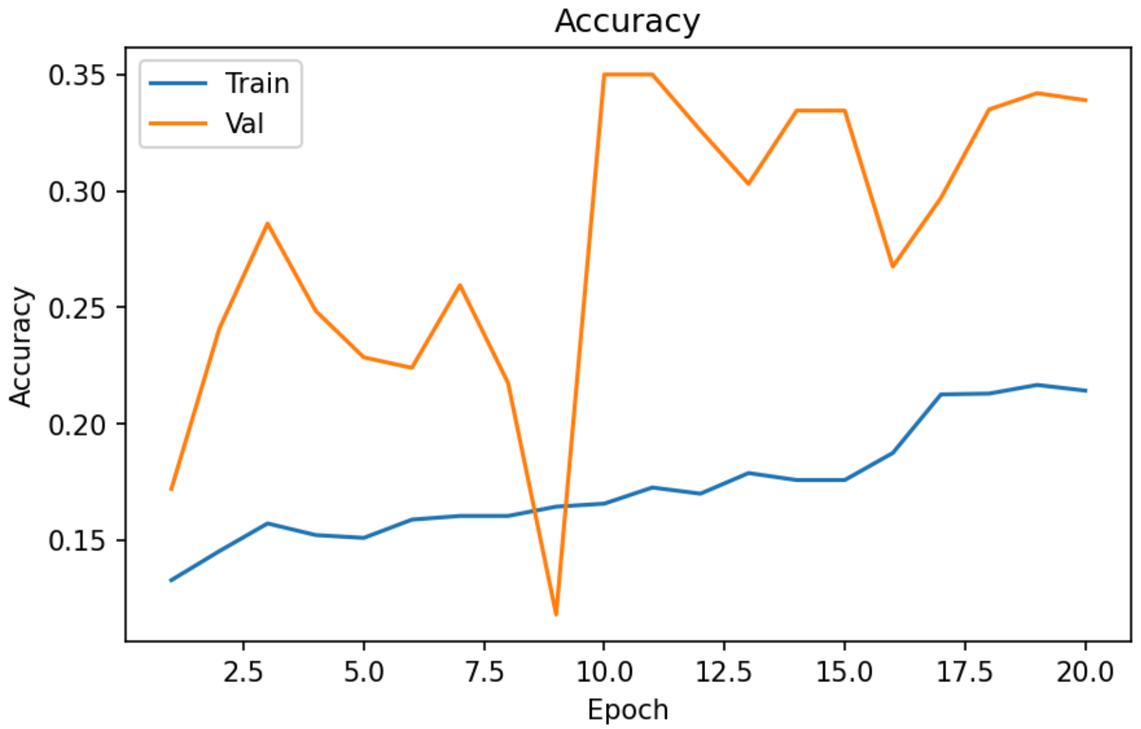


Model collapse. The model predicts "down" or "fist_moved" for almost everything: fist_moved → down: 1.00 palm_moved → down: 1.00 ,c → down: 0.95 ,ok → fist_moved: 0.74 ,Only palm (0.72) and down (1.00) recognized correctly. Average accuracy is only ~18% — barely above the 10% random baseline. The 4-million-parameter network is worse than logistic regression with 26× fewer parameters.

METHOD

Multi-Layer Perceptron

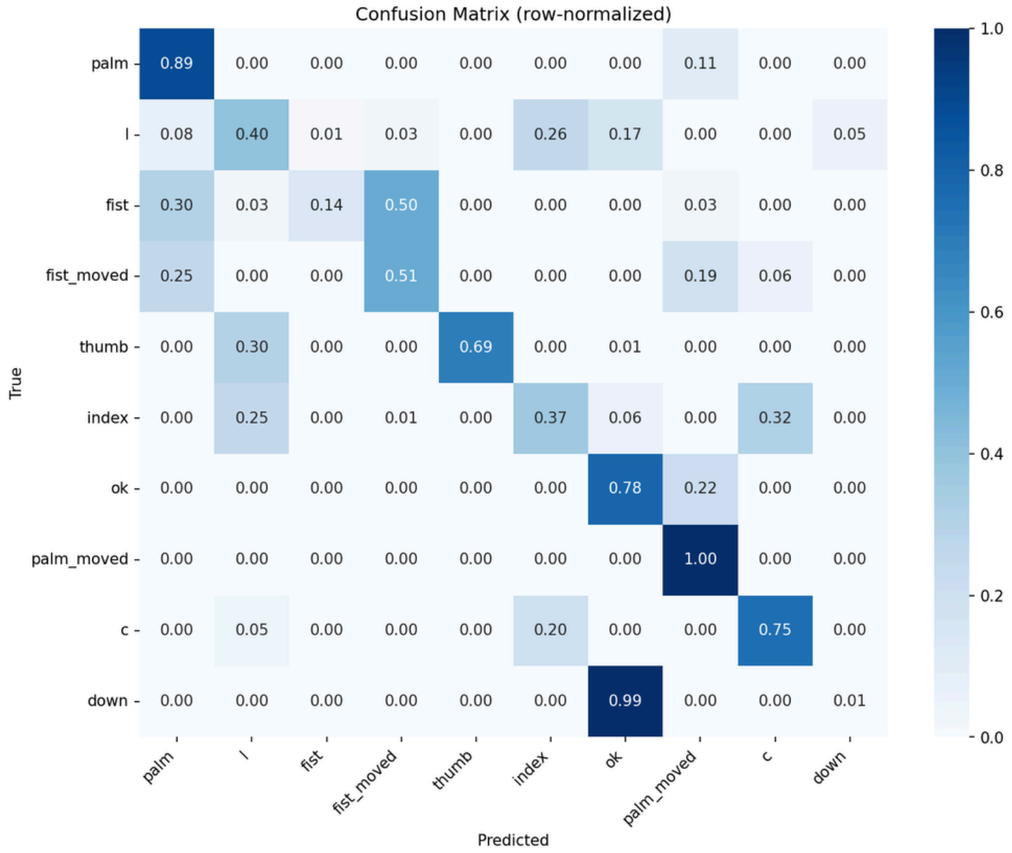




Loss stays near 2.1 (ln 10 ≈ 2.3 is random guessing). Train accuracy crawls from 0.13 to 0.21. Val jumps erratically between 0.12 and 0.35. The model is barely learning at all.

METHOD

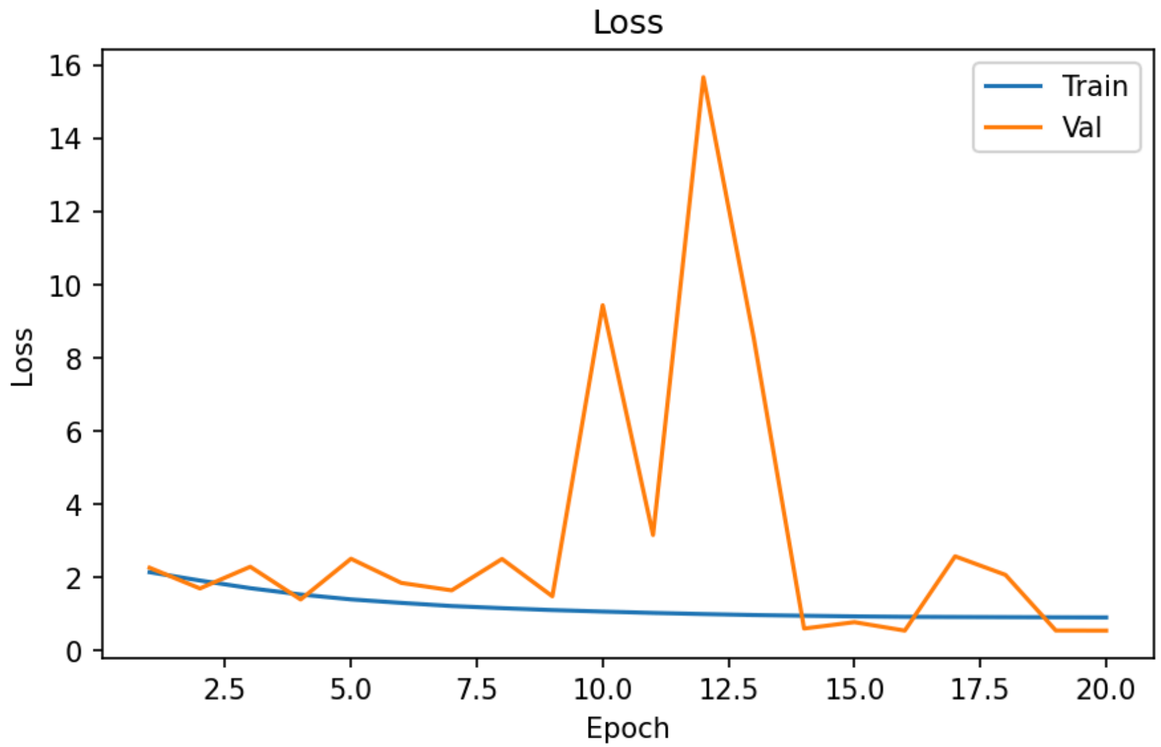
SmallCNN

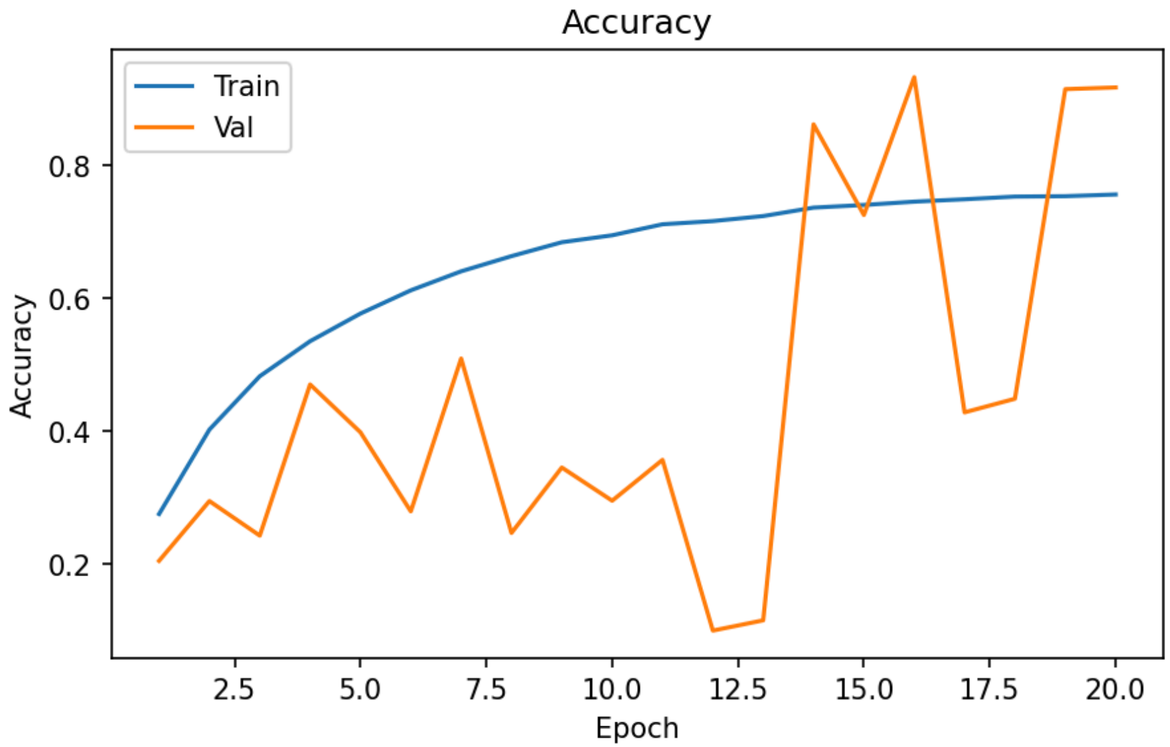


Mixed picture, average ~55%. Wins: palm_moved (1.00 perfect), palm (0.89), ok (0.78), c (0.75), thumb (0.69). Losses: down collapsed to 0.01 (almost all predictions go to ok), fist (0.14 confused with fist_moved 0.50), index (0.37 confused with l 0.25 and c 0.32). The model learned spatial structure (palm/palm_moved perfect) but only 5K parameters can't capture every gesture's nuances.

METHOD

SmallCNN





Train converges smoothly (loss 2 → 1, accuracy 0.27 → 0.75). Validation has a massive loss spike to 15.6 at epoch 12, val accuracy crashing to 0.1 then rocketing to 0.95. This is unstable training, likely because 5K parameters is too few small weight perturbations cause large prediction changes.

METHOD

Our Main Model: Custom CNN Architecture

Spatial size halves while channel count doubles at each block zooming out spatially, building richer features.

Input	Block 1	Block 2	Block 3	Block 4	AvgPool
1 × 128 × 128	32 × 64 × 64	64 × 32 × 32	128 × 16 × 16	256 × 8 × 8	256
Grayscale image	Conv + BN + ReLU + Pool	Conv + BN + ReLU + Pool	Conv + BN + ReLU + Pool	Conv + BN + ReLU + Pool	Spatial → feature vector

Classifier head:

`Linear(256→128) → ReLU → Dropout → Linear(128→10) → 10 gesture scores`

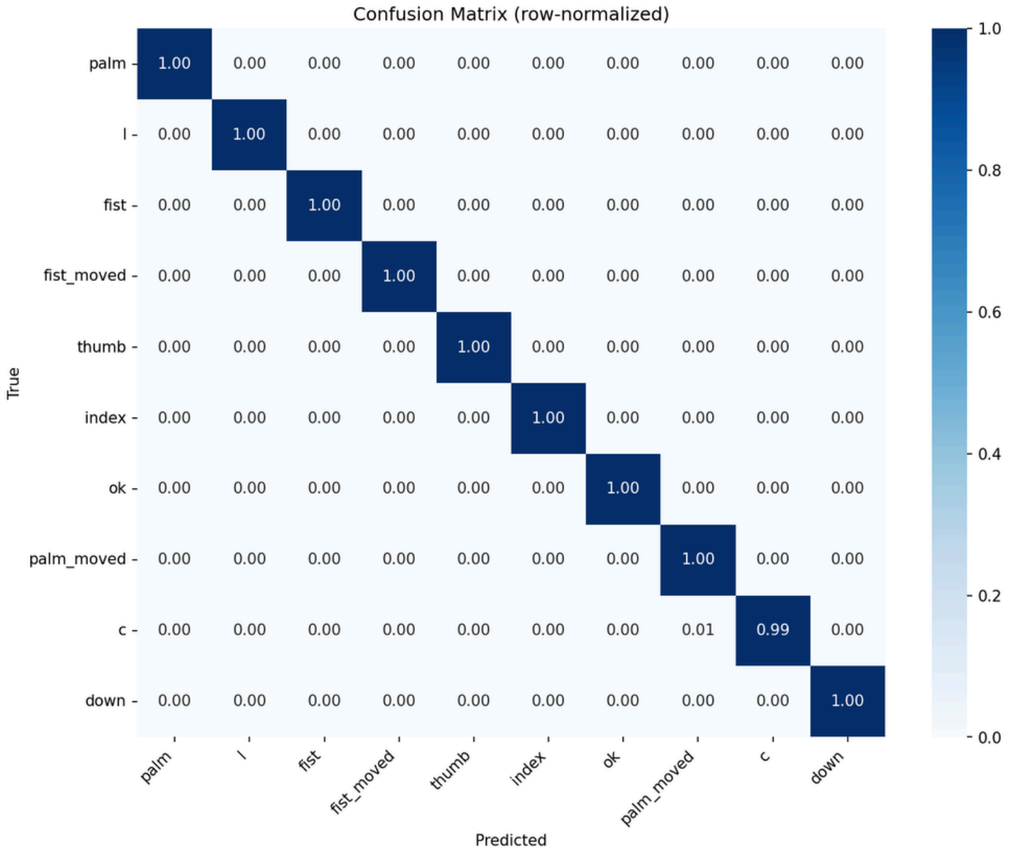
423,000

total trainable parameters

Big enough to capture real gesture features, small enough to train fast on a laptop GPU. Less than half the size of a typical ImageNet model.

METHOD

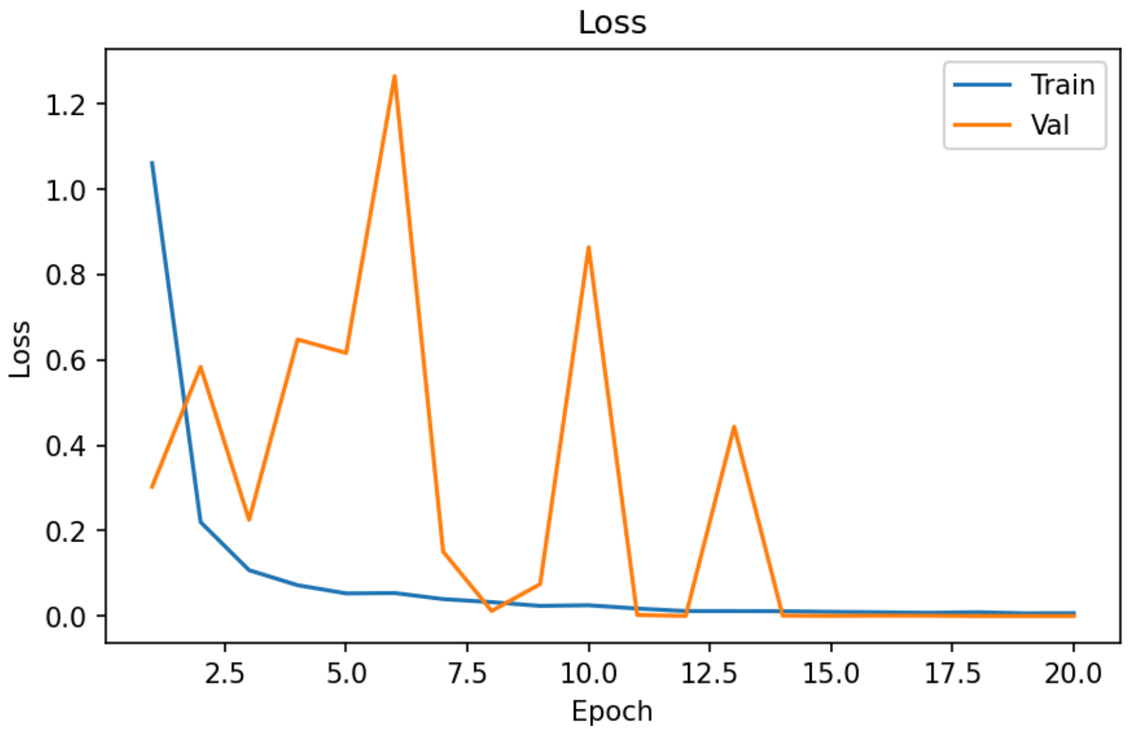
CNN

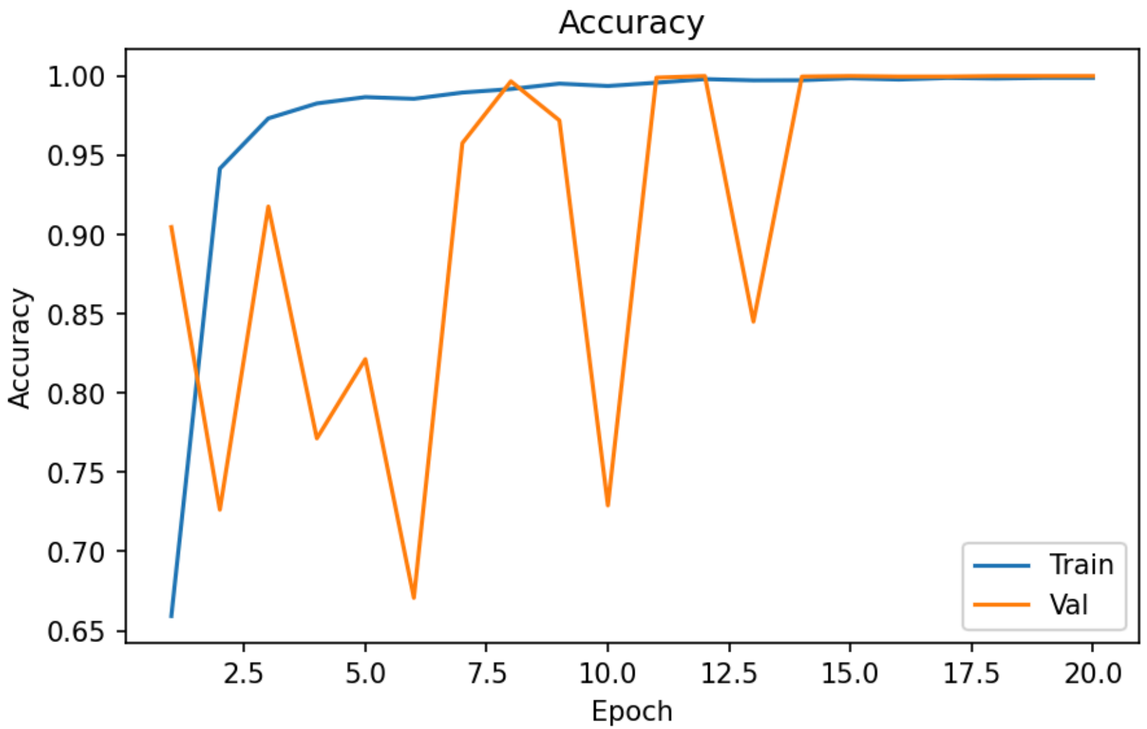


Near-perfect identity. Nine of ten classes hit 1.00 exactly. The only mistake in the entire test set is c → palm_moved at 0.01 . Overall test accuracy is essentially 99.9%.What's interesting about that single confusion: c and palm_moved both involve a curved, partially-open hand shape exactly the kind of geometric similarity you'd expect to trip up even a perfect-otherwise model. The model isn't randomly making mistakes; it's making the one mistake that makes physical sense.

METHOD

CNN

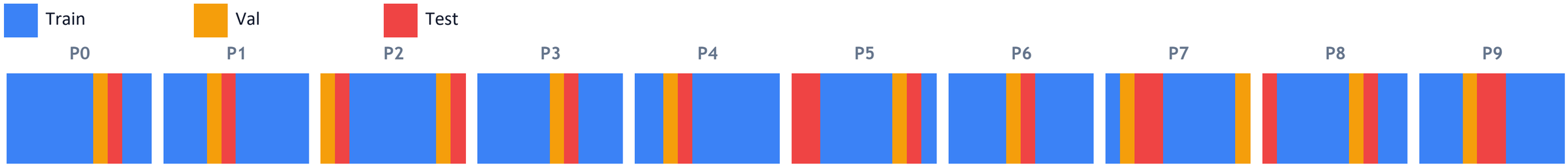




Train loss drops from 1.06 to near zero by epoch 5 and stays there. Train accuracy hits 100% around epoch 7 and never wavers. The model finds a near-perfect solution fast. While train is already memorizing the training set, val accuracy bounces because the model hasn't yet found weights that generalize even within the same people. The cosine learning-rate decay is what finally lets it settle.

The Random Split Approach

Take all 20,000 images, shuffle randomly, slice off 70 / 15 / 15.

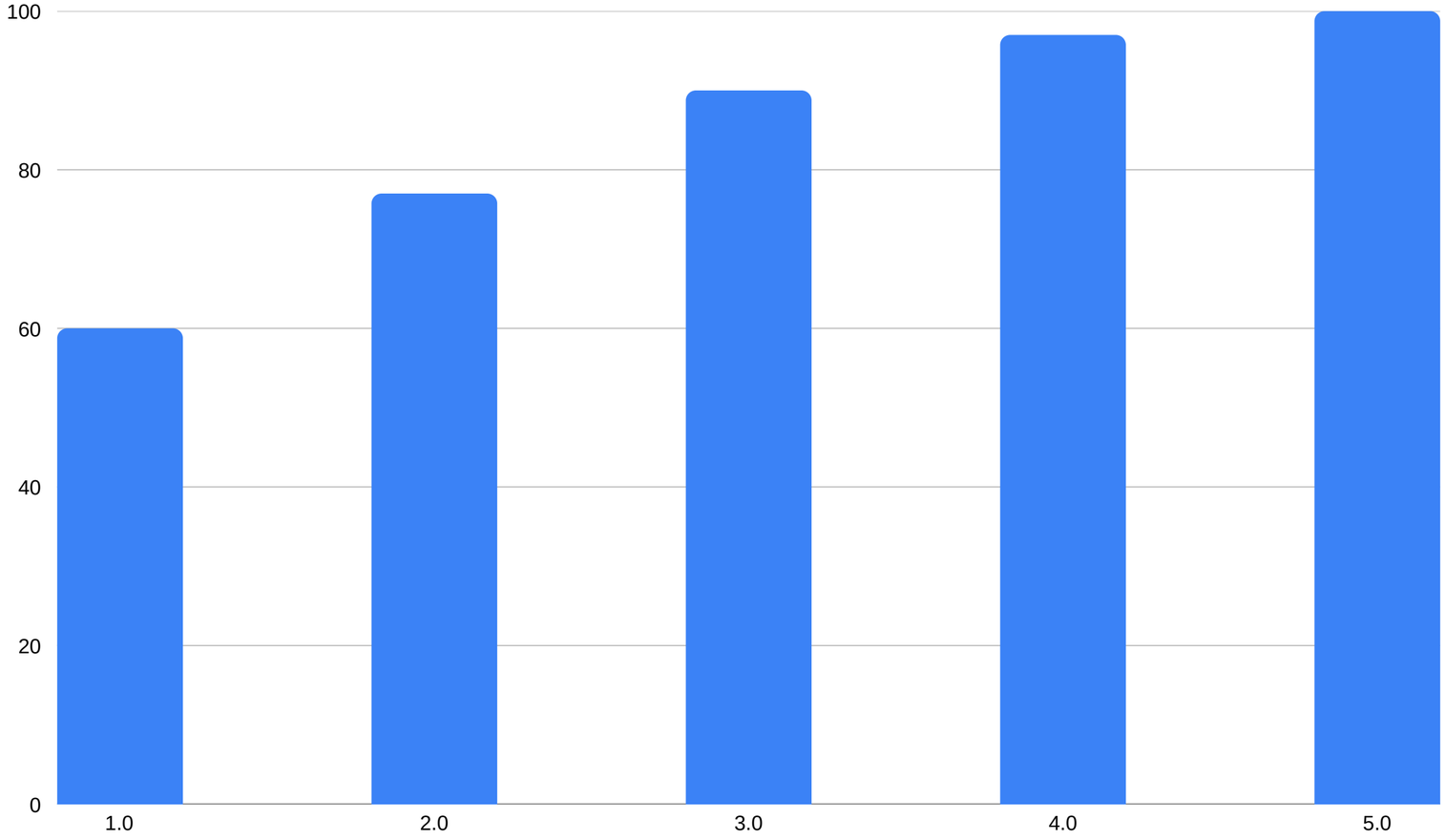


The problems

- Every person's hands end up in all three groups.
- Their fist might be in training while their palm is in testing.
- The model has already seen those exact hands during training just doing different gestures.
- The model is recognizing hands it memorized, not gestures it learned.
- This is how most papers and projects split data and it has a problem we'll unpack soon.

Results on the Random Split

Test accuracy of all five models



What to notice

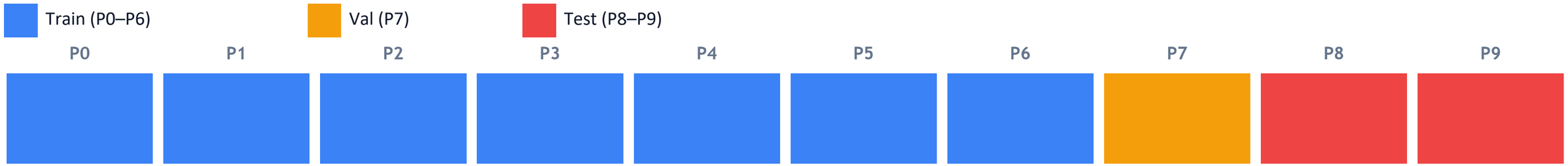
- Every step in the progression yields a measurable gain
- MLP has 4M parameters but loses to SmallCNN with only 5K (800× fewer)
- Convolutional structure matters way more than parameter count
- Custom CNN basically nails it

100% should is suspicious.

EVALUATION

The Subject Split Approach

Shuffle people, not images. Assign whole subjects to each set.

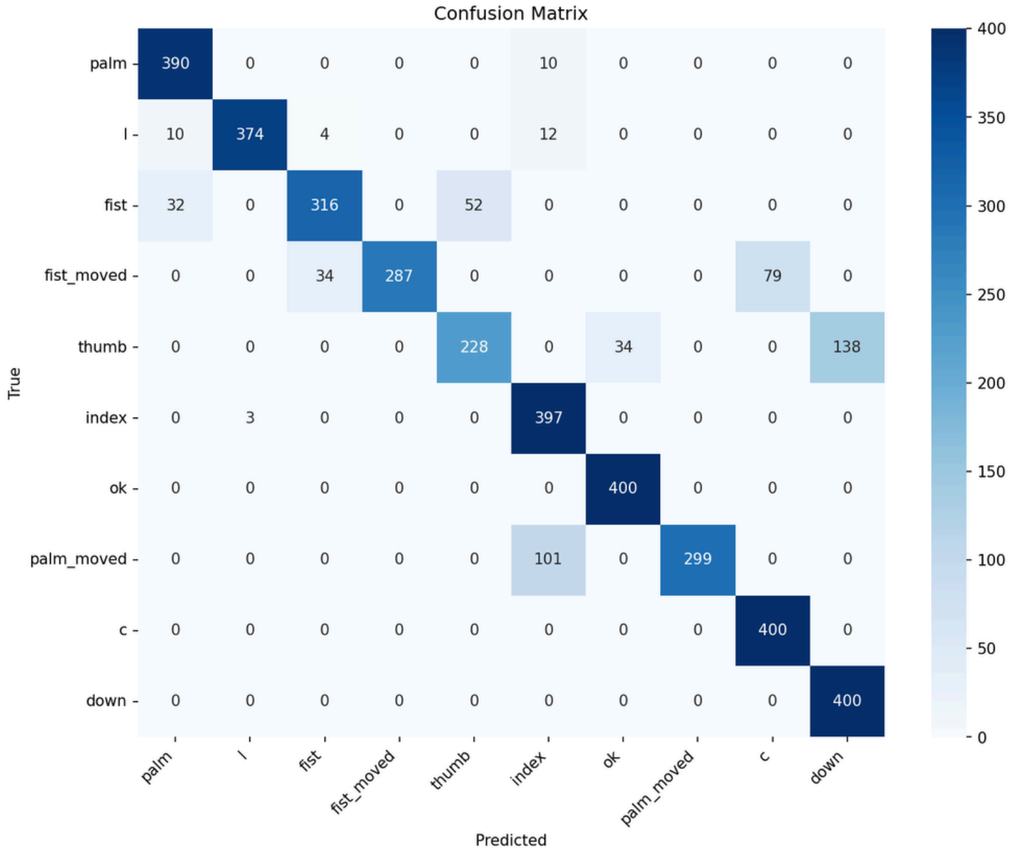


The crucial difference

- No person appears in more than one split.
- All of a single subject's images stay together in the same set.
- When tested, the model is seeing hands it has never seen before, in any form.
- This is exactly what real-world deployment looks like a new person walks up and tries to use the system.
- It's a much harder, much more honest test of whether the model actually learned gestures.

METHOD

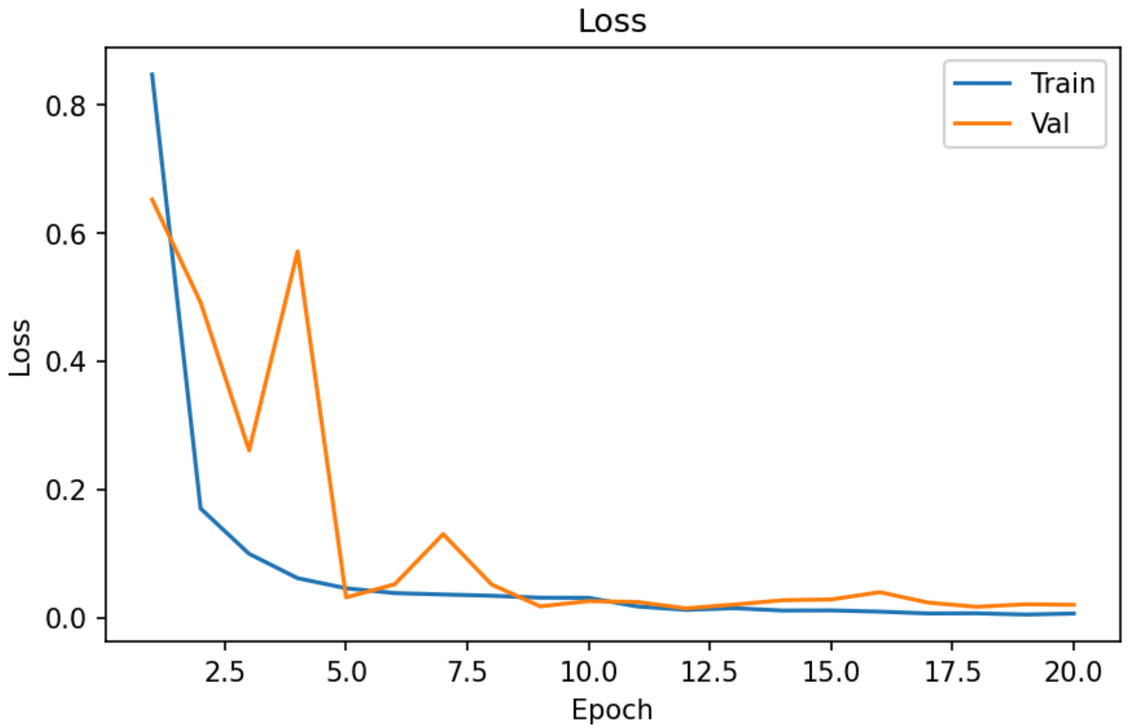
CNN(Subject Split)

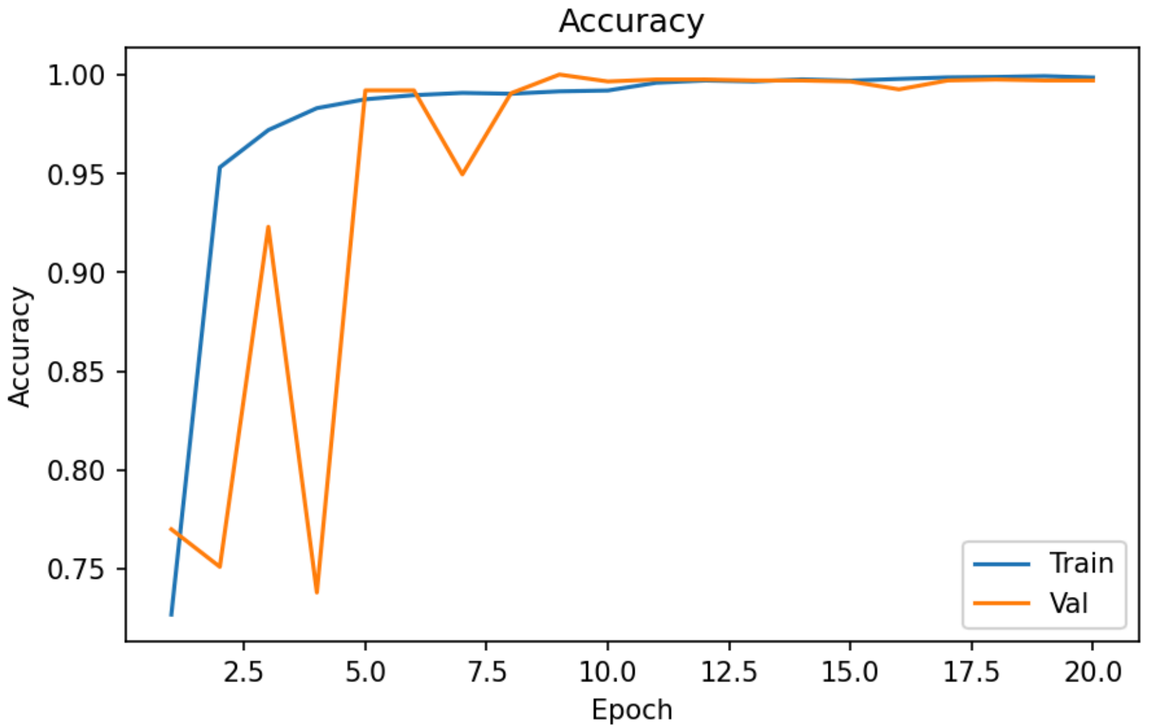


On the subject split, our CNN drops from 100% to 87% and the confusion matrix shows exactly where. Six gestures still nail it: ok, c, down, index, palm, and l. These have distinctive silhouettes that generalize across hands. The surprise is thumb at 57%, with a third of thumb gestures predicted as down. They both look like a single vertical digit. The model can't fake this anymore it has to actually distinguish similar shapes across people it has never seen.

METHOD

CNN(Subject Split)





Train converges by epoch 5. Val is bumpy for the first few epochs but locks in with train by epoch 8 and they stay together for the rest of training. No overfitting gap, no late-training drift. One subtle point: val accuracy reaches almost 100% while test is 87%

EVALUATION

Random vs Subject Split

Two splits, two fundamentally different questions about the model.

Random Split

ANSWERS

Can the model recognize gestures from people it has already seen?

- Shuffles images every person in train, val, and test
- Tends to produce very high accuracy numbers
- Standard approach in most introductory ML work
- Inflates apparent performance

Subject Split

ANSWERS

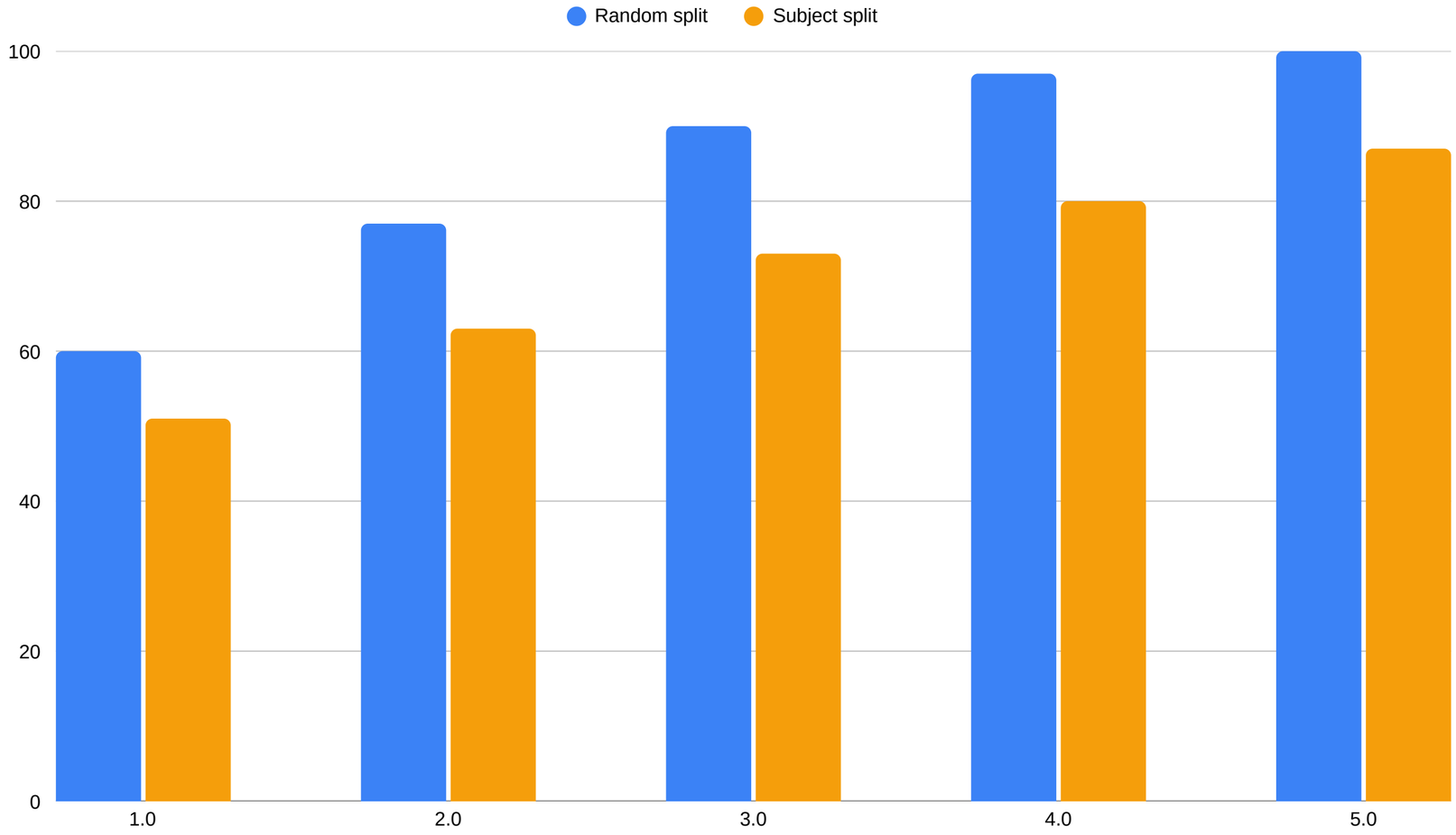
Can the model recognize gestures from completely new people?

- Shuffles people each person in only one split
- Produces lower but more realistic accuracy numbers
- Reflects what would happen in real-world deployment
- Honest measure of generalization

The gap between these methods is called data leakage and it has a large effect.

Results on the Subject Split

Same models, evaluated on people they've never seen before.



The drops

- Custom CNN: 100% → 87% (–13pp)
- SmallCNN: 97% → 80% (–17pp)
- MLP: 90% → 73% (–17pp)
- Logistic: 77% → 63% (–14pp)
- Linear: 60% → 51% (–9pp)

Deeper models had more to lose they could memorize more.

KEY INSIGHT

The Big Finding: Subject Leakage

13

percentage points

the real cost of subject leakage in our evaluation

Custom CNN: from ~100% on the random split down to ~87% on completely unseen people.

Why this happened

- Same person appeared in both training and testing
- Model could learn person-specific features (long fingers, wide palms)
- At test time it recognized the same person and used those shortcuts
- It wasn't learning gestures it was learning people

ANALOGY

Imagine a student who memorized which questions appear on the practice exam, then is shocked when those exact questions appear on the real exam.

Results

	PALM	L	FIST	FIST_MOVED	INDEX	OK	PALM_MOVED	C	DOWN	THUMB
F1	0.9375	0.9626769626769627	0.8381962864721485	0.8355167394468704	0.8630434782608696	.9592326139088729	0.8555078683834049	0.9101251422070534	0.8528784648187633	0.6705882352941176
PRECISION	0.9027777777777778	0.9920424403183024	0.8926553672316384	1.0	0.7634615384615384	0.9216589861751152	1.0	0.8350730688935282	0.7434944237918215	0.8142857142857143
RECALL	0.975	0.935	0.79	0.7175	0.9925	1.0	0.7475	1.0	1.0	0.57

ANALYSIS

Ablation Studies

Remove one component at a time, retrain, see what breaks.

Configuration	Accuracy	Effect	Verdict
Full Custom CNN	100%	—	*Baseline*
No BatchNorm	99.4%	–0.6pp + unstable training	*Important*
No augmentation	99.9%	essentially no effect	*Minor here, matters more on subject split*
No dropout	99.95%	no effect	*Decorative (BatchNorm already regularizes)*

What we learned

- BatchNorm earned its place without it, training was wobbly and accuracy dropped.
- Dropout was decorative BatchNorm already provides enough regularization for this task.
- These ablations tell us why our model works, not just that it works.

ANALYSIS

Fine Tuning

What we searched

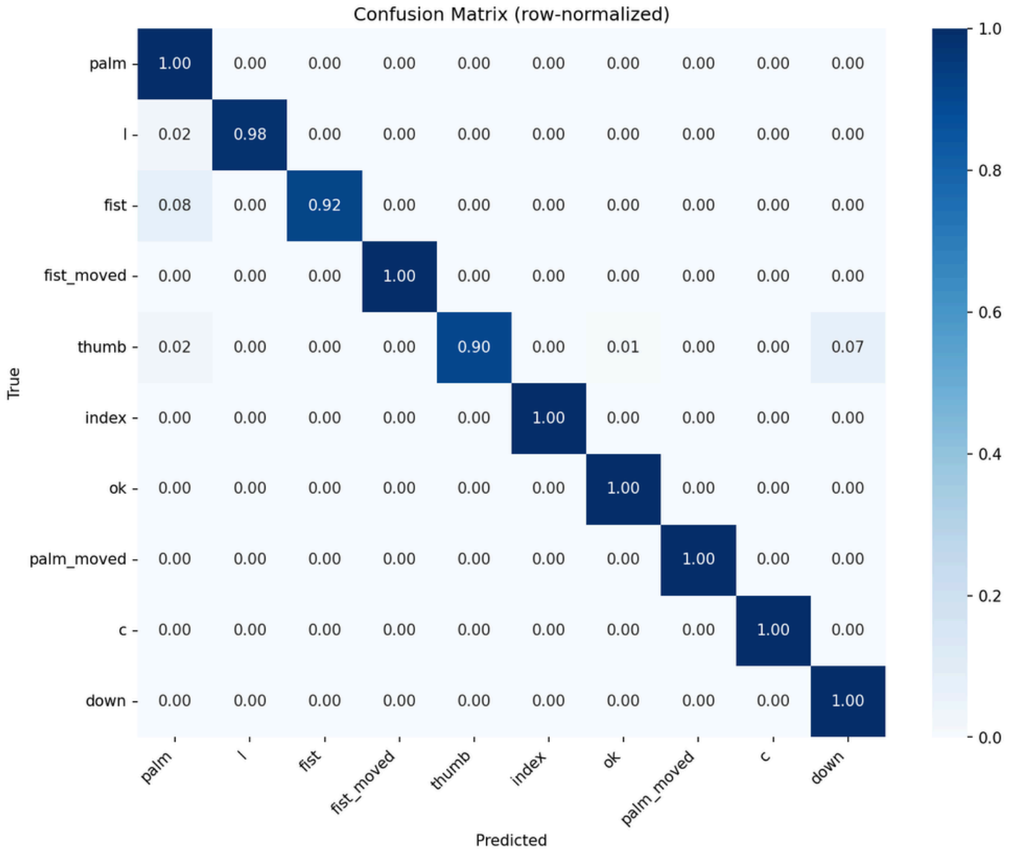
- Hyperparameter :
  - Learning Rate
  - Dropout
  - Weight decay
  - Optimizer

What we found

- Hyperparameter :
  - Learning Rate : 1e-3
  - Dropout : 0.3
  - Weight decay : 1e-4
  - Optimizer : Adam

METHOD

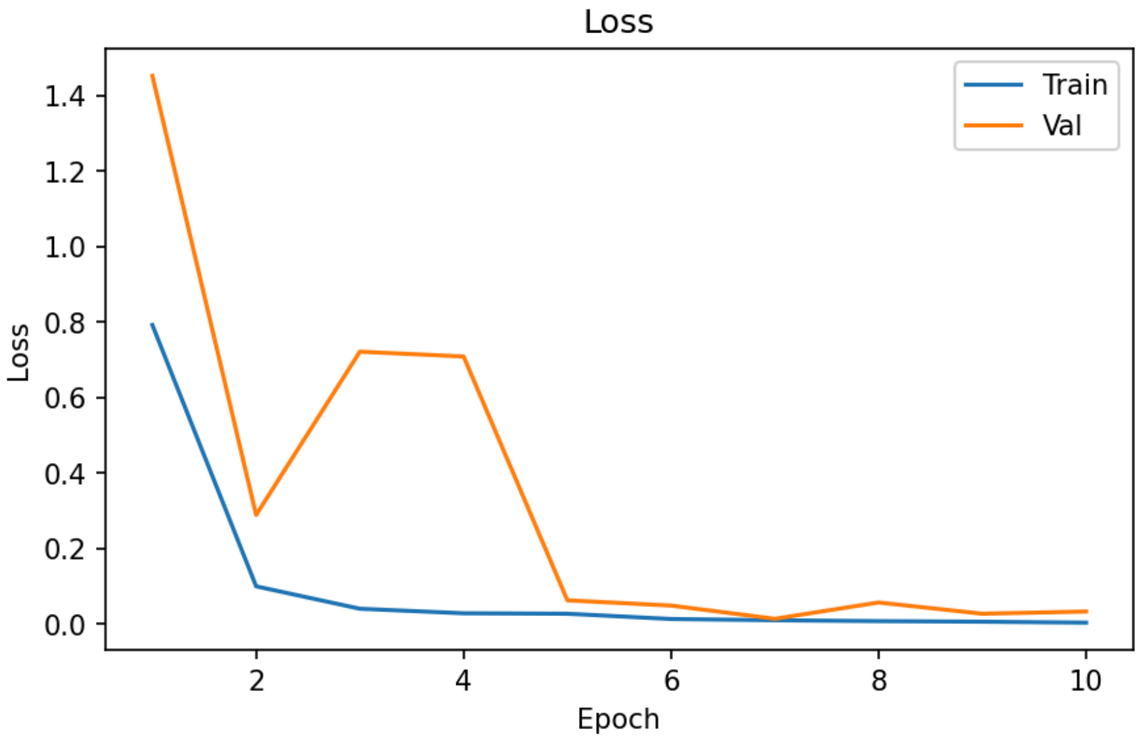
CNN(Fined tuned)

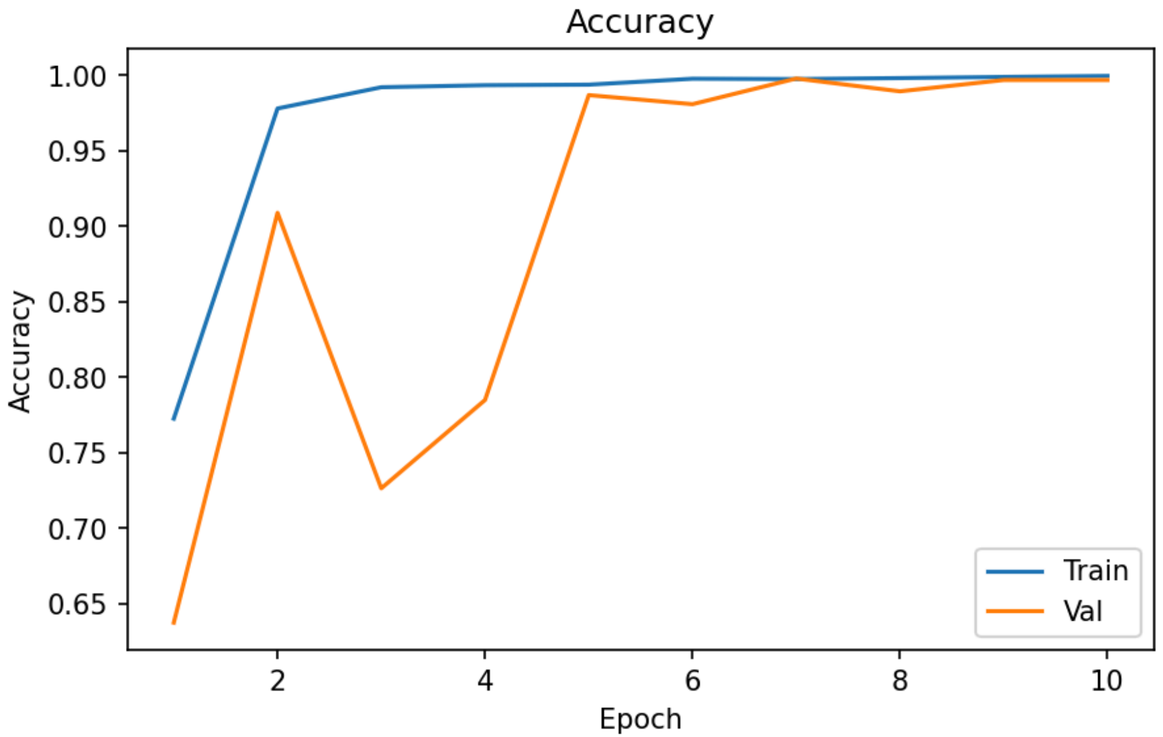


The right hyperparameters made the model better with now a 98% accuracy but the difference between the random split and our tuned Subject Split is that this model should be better at generalizing gestures between people and not just memerizing

METHOD

CNN(Fined tuned)



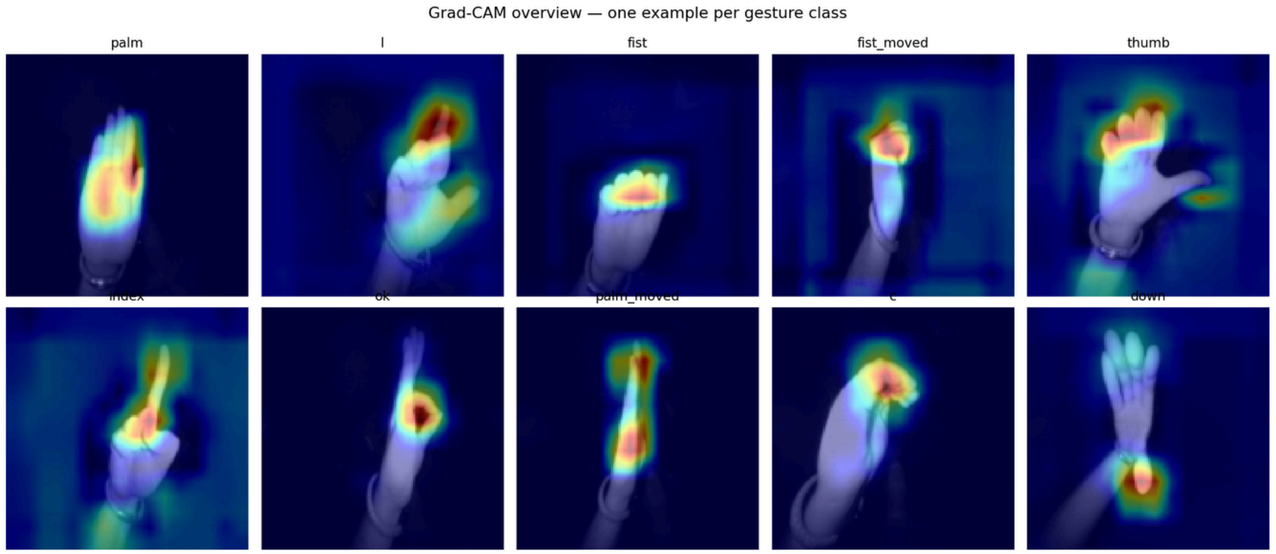


Tuning didn't just improve accuracy it made training faster, more stable, and more trustworthy

ANALYSIS

Looking Inside the Model

A visualization technique that highlights which regions of an image influenced the model's prediction.



What we found

- Heatmaps consistently focused on the hand, not the background.
- Model learned meaningful features, not visual shortcuts.
- Hardest pairs: palm vs palm_moved, fist vs fist_moved (as predicted).
- Easiest: thumb, L, down distinctive shapes.
- Even on misclassifications, the model still attended to the hand. Errors are semantic, not localization failures.

The CNN is doing the right kind of work looking at hands, not artifacts.

CONCLUSIONS

What We Learned & What's Next

WHAT WE ACCOMPLISHED

- Custom CNN reaching ~98% honest test accuracy on unseen people
- Identified BatchNorm as the most important architectural choice
- Quantified subject leakage random split inflates results by ~13pp
- Grad-CAM confirmed the model attends to the right features

WHAT'S NEXT

- Transfer learning from pretrained models (likely past 90% on subject split)
- Temporal modeling for gesture sequences in video
- More diverse data varied people, lighting, backgrounds
- Domain adaptation from near-IR to standard RGB webcam input
- Cross-dataset evaluation for stronger generalization claims